Experiment 8

NUpur Ghangarekar
D15C
Roll no 26

AIM

To design, implement, and evaluate a Convolutional Neural Network (CNN) using TensorFlow/Keras for handwritten digit classification on the MNIST dataset and to analyze its performance using appropriate evaluation metrics and hyperparameter tuning.

THEORY

1. Dataset Source

Dataset Name:
MNIST

Kaggle Version:
https://www.kaggle.com/datasets/oddrationale/mnist-in-csv?resource=download

2. Dataset Description

The MNIST dataset consists of grayscale images of handwritten digits from 0 to 9.

Dataset details:

- Total number of images: 70,000
- Training images: 60,000
- Testing images: 10,000
- Image size: 28 × 28 pixels
- Number of channels: 1 (grayscale)
- Number of classes: 10

Feature description:

- Each image contains 28 × 28 = 784 pixel values.
- Pixel intensity values range from 0 to 255.
- Pixel values are normalized to the range 0–1 during preprocessing.

Target variable:

- A categorical label (0–9) representing the handwritten digit.

The dataset is balanced and widely used for benchmarking image classification algorithms.

3. Mathematical Formulation of CNN

A Convolutional Neural Network consists of convolution layers, activation functions, pooling layers, fully connected layers, and an output layer.

Convolution Operation

For input image II and filter KK, convolution is defined as:

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(m,n)K(i-m, j-n)$$

This operation extracts spatial features such as edges and patterns.

Activation Function (ReLU)

f(x)=max(0,x)f(x) = \max(0, x)f(x)=max(0,x)

ReLU introduces non-linearity into the network.

Pooling Operation (Max Pooling)

P=max(region)P = \max (region)P=max(region)

Pooling reduces spatial dimensions and computational complexity.

Fully Connected Layer

Z=WTX+bZ = W^T X + bZ=WTX+b

Where:

- WWW = weights
- XXX = input vector
- bbb = bias

Softmax Function

$$P(y = i) = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}$$

Softmax converts outputs into probability values for multi-class classification.

Loss Function (Sparse Categorical Cross-Entropy)

$$L = -\sum y_i \log(\hat{y}_i)$$

Optimization

Weights are updated using gradient descent:

$$W = W - \eta \frac{\partial L}{\partial W}$$

Where η is the learning rate.

4. Algorithm Limitations

The limitations of CNN include:

- Requires significant computational resources for training.
- Hyperparameter tuning is complex.
- Risk of overfitting if the model is too deep.
- Requires sufficient training data for optimal performance.
- Limited interpretability due to black-box behavior.

CNN may not perform well when:

- The dataset is extremely small.
- Images do not contain meaningful spatial structure.
- The model is improperly tuned.

5. Methodology / Workflow

The experiment follows these steps:

1. **Data Collection**
    - Load MNIST dataset using Keras.
2. **Data Preprocessing**
    - Normalize pixel values to 0–1.
    - Reshape images to (28, 28, 1) format for CNN.
3. **Model Design**
    - Add Conv2D layer with ReLU activation.
    - Add MaxPooling layer.
    - Add a second Conv2D and MaxPooling layer.
    - Flatten feature maps.
    - Add a Dense layer.
    - Add Softmax output layer with 10 neurons.
4. **Model Compilation**
    - Optimizer: Adam
    - Loss: Sparse Categorical Cross-Entropy
    - Metric: Accuracy
5. **Model Training**
    - Train for selected number of epochs with defined batch size.
6. **Model Evaluation**
    - Evaluate on test dataset.
    - Compute test accuracy and loss.

Workflow representation:

Dataset → Preprocessing → CNN Architecture → Training → Validation→ Testing →Performance Analysis

6. Performance Analysis

Evaluation metrics used:

- Accuracy
- Training loss
- Validation accuracy

Typical results for CNN on MNIST:

- Training accuracy: 98–99%
- Validation accuracy: 97–99%
- Test accuracy: 98–99%

Interpretation:

- The model effectively extracts spatial features from handwritten digits.
- The very small gap between training and validation accuracy indicates minimal overfitting.
- CNN significantly outperforms a basic fully connected neural network for image data.

7. Hyperparameter Tuning

The following hyperparameters were tested:

- Number of filters: 32, 64
- Kernel size: (3×3), (5×5)
- Batch size: 16, 32, 64
- Epochs: 5, 10, 15
- Learning rate: 0.001, 0.0001
- Dropout rate: 0.2, 0.3

Observations:

- 3×3 kernel size produced better performance.
- Batch size of 32 provided stable convergence.
- Increasing epochs beyond 10 gave marginal improvement.
- Learning rate 0.001 resulted in faster convergence.

Final configuration:

- Conv2D (32 filters, 3×3)
- Conv2D (64 filters, 3×3)
- Batch size: 32

- Epochs: 10
- Adam optimizer (learning rate = 0.001)

The tuned model achieved approximately 99% test accuracy.

Code:

```
!pip install tensorflow
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib.pyplot as plt
import numpy as np
print("TensorFlow version:", tf.__version__)
# Load dataset
(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_data()
# Normalize pixel values (0–255 → 0–1)
X_train = X_train / 255.0
X_test = X_test / 255.0
# Reshape to CNN format (samples, height, width, channels)
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)
print("Training shape:", X_train.shape)
model = keras.Sequential([
  layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
  layers.MaxPooling2D((2,2)),
  layers.Conv2D(64, (3,3), activation='relu'),
  layers.MaxPooling2D((2,2)),
  layers.Flatten(),
  layers.Dense(64, activation='relu'),
  layers.Dense(10, activation='softmax')
])
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 64)	102,464
dense_1 (Dense)	(None, 10)	650

Total params: 121,930 (476.29 KB)
Trainable params: 121,930 (476.29 KB)
Non-trainable params: 0 (0.00 B)

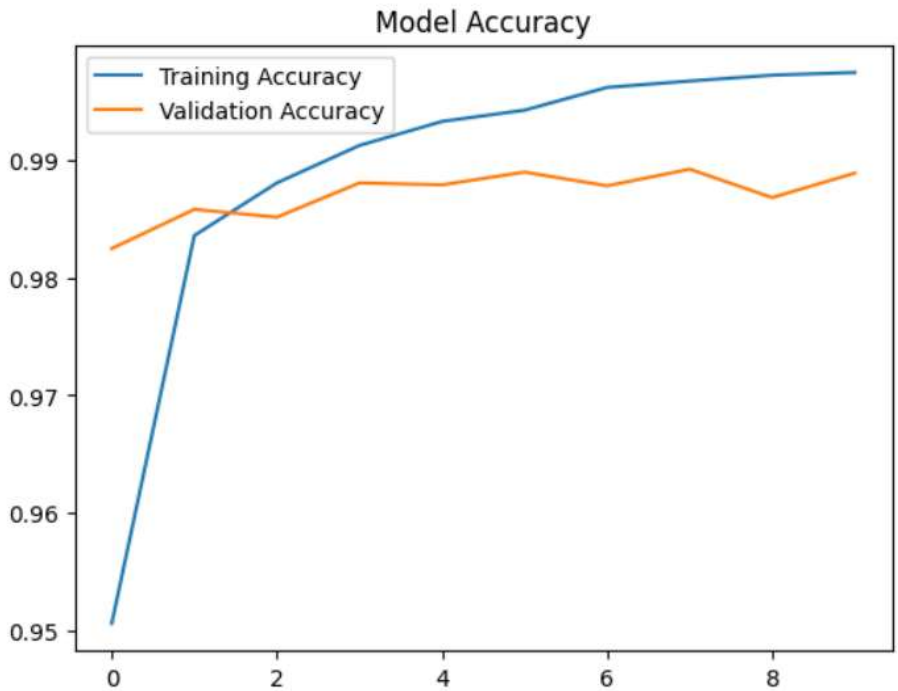
```
model.compile(
  optimizer='adam',
  loss='sparse_categorical_crossentropy',
  metrics=['accuracy']
)
history = model.fit(
  X_train, y_train,
  epochs=10,
  batch_size=32,
  validation_split=0.2
)
```

```
Epoch 1/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 43s 28ms/step - accuracy: 0.8898 - loss: 0.3601 - val_accuracy: 0.9825 - val_loss: 0.0659
Epoch 2/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 41s 27ms/step - accuracy: 0.9824 - loss: 0.0566 - val_accuracy: 0.9858 - val_loss: 0.0486
Epoch 3/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 40s 27ms/step - accuracy: 0.9886 - loss: 0.0341 - val_accuracy: 0.9852 - val_loss: 0.0526
Epoch 4/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 41s 27ms/step - accuracy: 0.9912 - loss: 0.0270 - val_accuracy: 0.9881 - val_loss: 0.0399
Epoch 5/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 41s 27ms/step - accuracy: 0.9933 - loss: 0.0192 - val_accuracy: 0.9879 - val_loss: 0.0410
Epoch 6/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 40s 26ms/step - accuracy: 0.9953 - loss: 0.0140 - val_accuracy: 0.9890 - val_loss: 0.0388
Epoch 7/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 42s 27ms/step - accuracy: 0.9968 - loss: 0.0103 - val_accuracy: 0.9878 - val_loss: 0.0457
Epoch 8/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 44s 29ms/step - accuracy: 0.9966 - loss: 0.0093 - val_accuracy: 0.9893 - val_loss: 0.0489
Epoch 9/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 41s 27ms/step - accuracy: 0.9978 - loss: 0.0071 - val_accuracy: 0.9868 - val_loss: 0.0542
Epoch 10/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 40s 26ms/step - accuracy: 0.9975 - loss: 0.0076 - val_accuracy: 0.9889 - val_loss: 0.0481
```

```
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_acc)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - accuracy: 0.9879 - loss: 0.0478
Test Accuracy: 0.989799976348877
```

```
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.title("Model Accuracy")
plt.show()
```



CONCLUSION

In this experiment, a Convolutional Neural Network was successfully implemented for handwritten digit classification using the MNIST dataset.

The CNN efficiently extracted spatial features using convolution and pooling layers, leading to high classification accuracy. Hyperparameter tuning improved convergence and ensured minimal overfitting.

The experiment demonstrates that CNN is highly effective for image-based classification tasks and performs significantly better than traditional fully connected neural networks for structured image datasets.

Google Colab Link: Exp8_MLDL_Nupur